

Vector Semantics and Embeddings

NLP - Week#04

Distributional Hypothesis

"You shall know a word by the company it keeps." - J.R. Firth (1957)

Words that occur in similar contexts tend to have similar meanings. This distributional hypothesis, first observed by 1950s linguists, reveals that meaning difference corresponds to difference in linguistic environments.

Example: oculist and eye-doctor both appear near words like "eye" and "examined" - similar distribution predicts similar meaning.

This fundamental insight enables computers to understand meaning mathematically, forming the theoretical foundation for modern word embeddings like Word2Vec, BERT, and GPT.

Cosine for measuring similarity

Whether using sparse or dense vectors, word and document similarities are computed by some function of the dot product between vectors. The cosine of two vectors—a normalized dot product—is the most popular such metric.

The cosine similarity metric between two vectors $\mathbf{v}$ and $\mathbf{w}$ thus can be computed as:

$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .017$$

$$\cos(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

The model decides that *information* is way closer to *digital* than it is to *cherry*, a result that seems sensible. Fig. 6.8 shows a visualization.

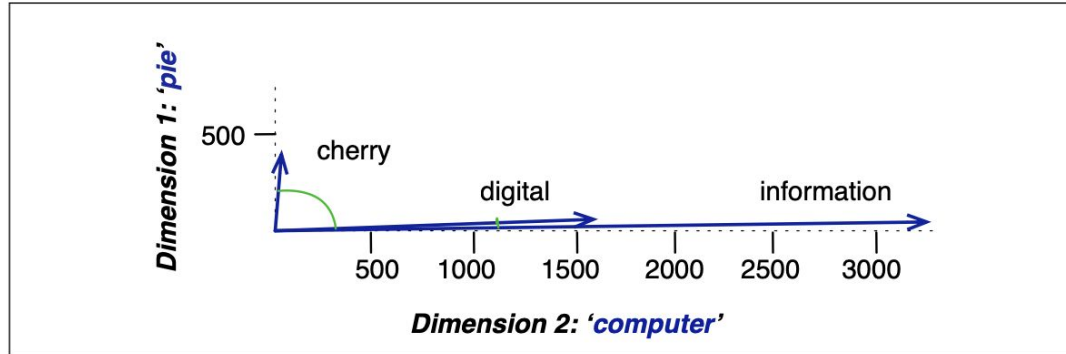


Figure 6.8 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. Note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0°); the cosine of all other angles is less than 1.

Task

Given two vectors representing word embeddings in a 2D semantic space:

Vector A = [4, 3] (representing the word "king")

Vector B = [1, 2] (representing the word "queen")

Compute similarity score using cosine similarity

Vector Semantics Embeddings

The distributional hypothesis is computationally implemented through vector semantics, which learns representations of word meanings (called embeddings) directly from word distributions in texts.

Static embeddings capture meaning from overall distribution patterns across a corpus, creating fixed vector representations for each word. These embeddings form the foundation for more powerful **dynamic/contextualized embeddings** like BERT, which adapt word representations based on specific sentence context.

Traditional ways to covert a word into vector

Problems:

- No Natural Similarity: "cat" and "dog" are equally different from "cat" and "philosophy"
- Data Sparsity: Millions of words, no relationships between them
- No Semantic Relationships: Can't capture that cats and dogs are both pets, animals

Solution: Vector Semantics

- Represent words as vectors (lists of numbers) in a mathematical space

Simple Example: 2D Word Vectors

Let's imagine we represent words using only 2 dimensions:
"Animal-ness" and "Food-ness"

Let's draw
these vectors
in a 2D
space

```
# Words represented as [animal_score, food_score]
```

```
cat = [0.9, 0.1] # Very animal, not food
```

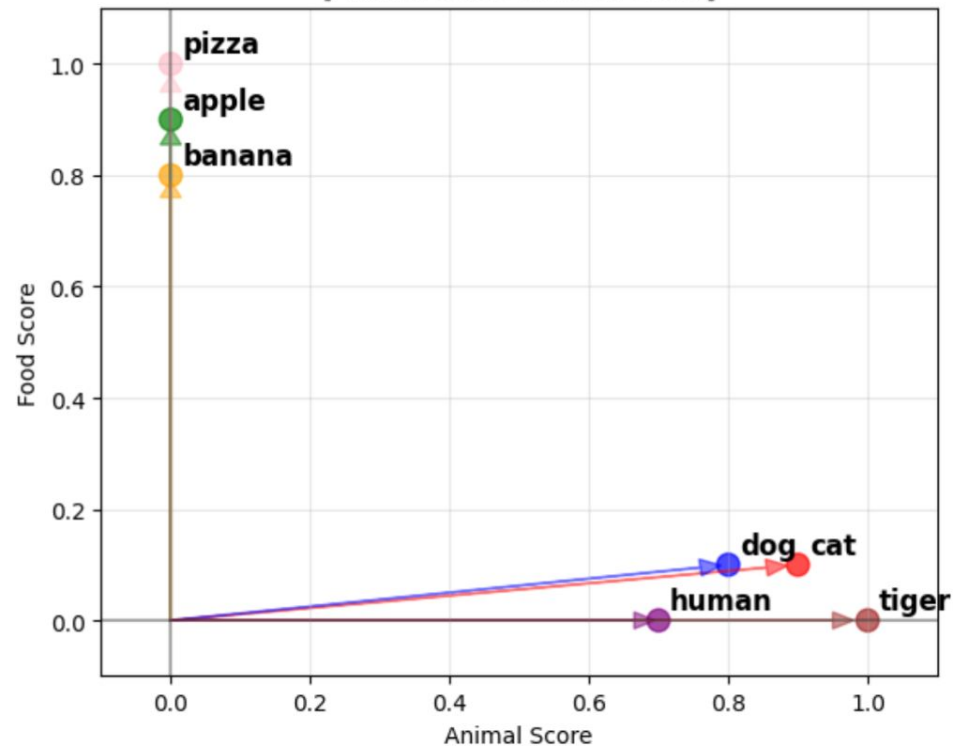
```
dog = [0.8, 0.1] # Very animal, not food
```

```
apple = [0.0, 0.9] # Not animal, very food
```

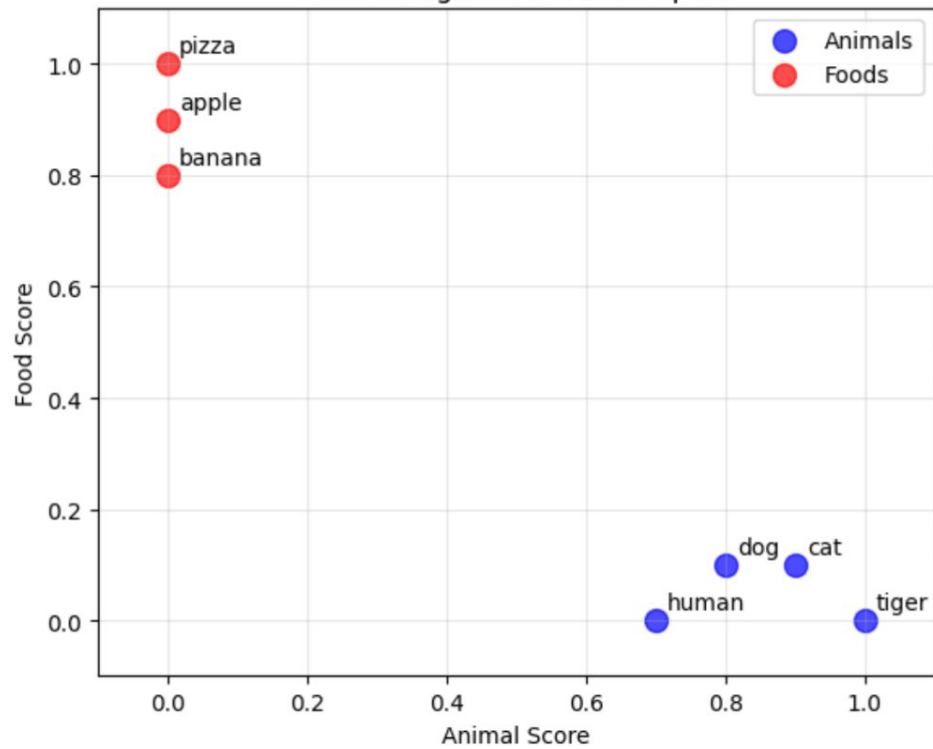
```
banana = [0.0, 0.8] # Not animal, very food
```

```
human = [0.7, 0.0] # Animal, not food
```

2D Word Vector Space
[Animal Score vs Food Score]



Word Categories in Vector Space



Distributed Hypothesis

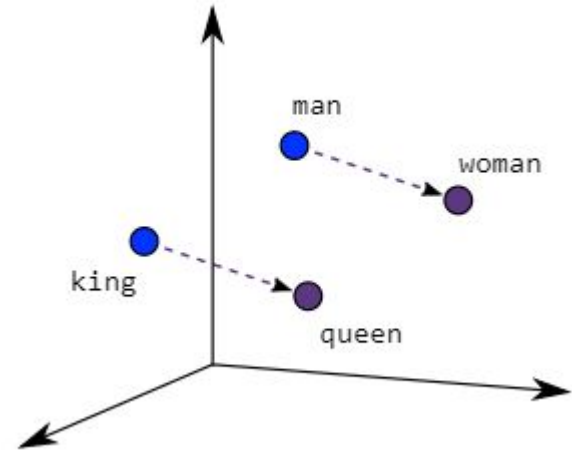
The hypothesis is most famously summarized by the linguist John Rupert Firth in 1957:

- "You shall know a word by the company it keeps."

This means that the linguistic context of a word (its "distribution" in a large corpus of text) provides a powerful representation of its semantic content.

What is Word2Vec?

Word2Vec creates vectors of the words that are distributed numerical representations of word features – these word features could comprise of words that represent the context of the individual words present in our vocabulary. Word embeddings eventually help in establishing the association of a word with another similar meaning word through the created vectors.



Why are word embeddings needed?

- **They Capture Meaning and Relationships**

- This is the most important reason. Embeddings place words with similar meanings close to each other in the vector space. The famous example is the vector algebra:

■ King - Man + Woman ≈ Queen

- **They Provide Dense, Efficient Representations**

- Instead of a 100,000-dimensional one-hot vector, an embedding might represent each word with only 100-300 dense (mostly non-zero) numbers. This is far more computationally efficient for machine learning models.

- **They Allow Models to Generalize**

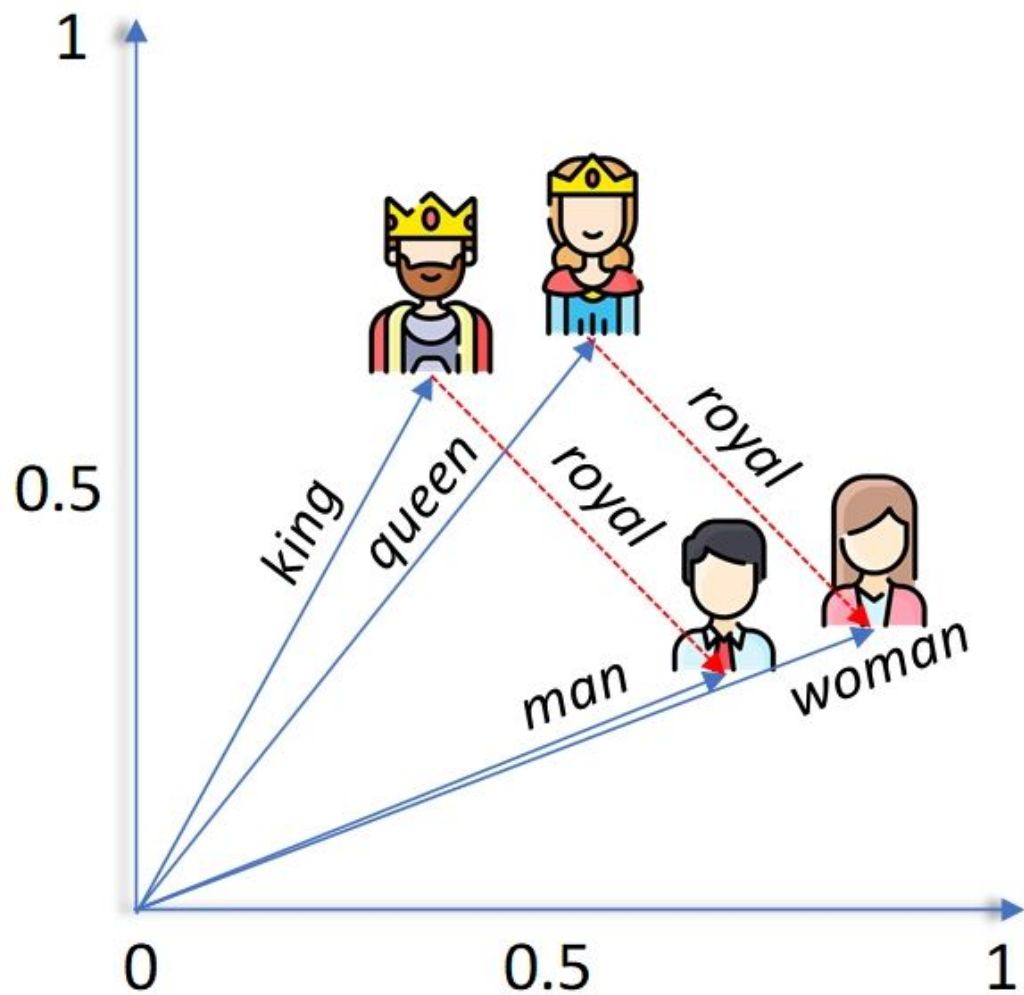
- Because similar words have similar vectors, if a model learns that "good" is often followed by "movie" in a review, it can naturally generalize this to "great" or "excellent" because their embedding vectors are nearby. With one-hot encoding, this is impossible.

- **They are a Form of Transfer Learning**

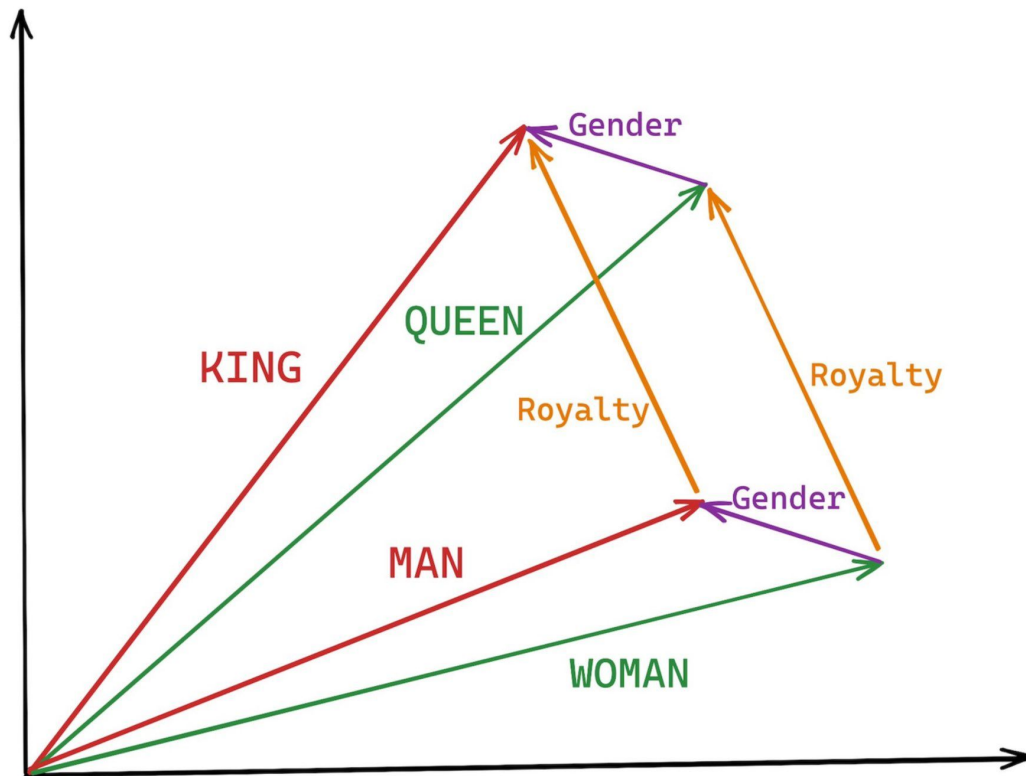
- You can use pre-trained embeddings (like Word2Vec, GloVe, or FastText) that have been trained on massive datasets (e.g., all of Wikipedia). This means your model doesn't have to learn language from scratch on your small dataset; it can start with a rich, pre-built understanding of word meanings.

Word2Vec

- Capture semantic & syntactic relationships
- Low dimension
- Dense vectors (not sparse) therefore can avoid the overfitting problem
- Models generalize to unseen words/data.
- Can leverage pre-trained knowledge.
- linear regularity
 - $\text{king} - \text{man} + \text{woman} \approx \text{queen}$



Embedding Space!



These word representations are also the first example in this book of representation learning, automatically learning useful representations of the input text. Finding such self-supervised ways to learn representations of the input, instead of creating representations by hand via feature engineering, is an important focus of NLP research (Bengio et al., 2013).

Static Embeddings

The word2vec methods are fast, efficient to train, and easily available online with code and pretrained embeddings. Word2vec embeddings are static embeddings, meaning that the method learns one fixed embedding for each word in the vocabulary.

Dynamic Embeddings:

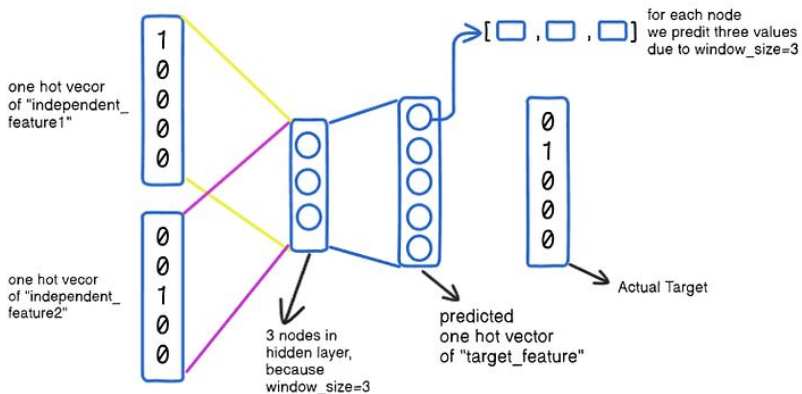
A methods for learning dynamic contextual embeddings like the popular BERT or ELMO representations, in which the vector for each word is different in different contexts.

Examples of Static & Dynamic Embeddings

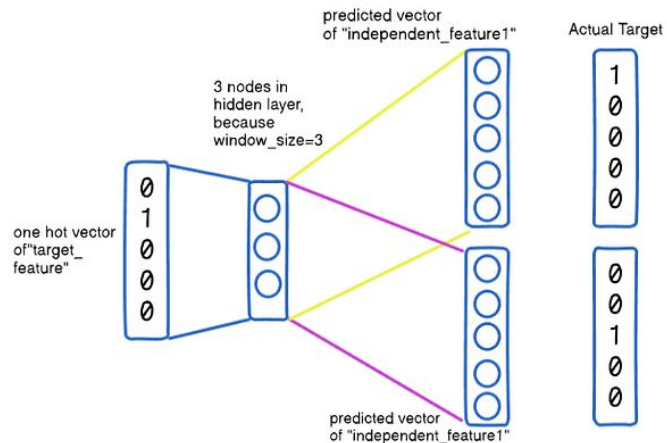
Word2Vec

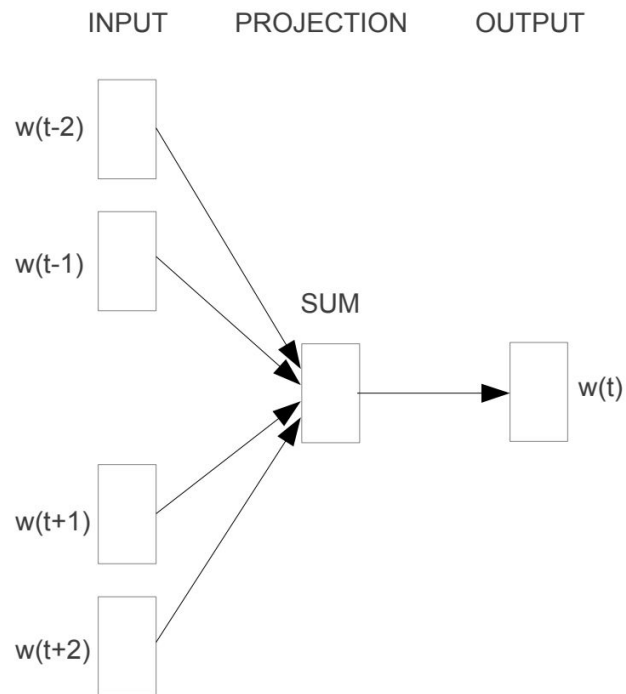
Word2vec

CBOW

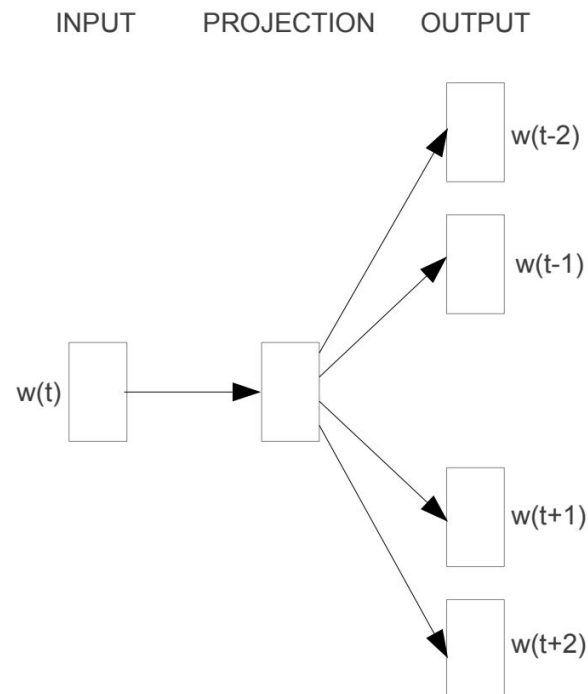


Skip-gram





CBOW



Skip-gram

Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

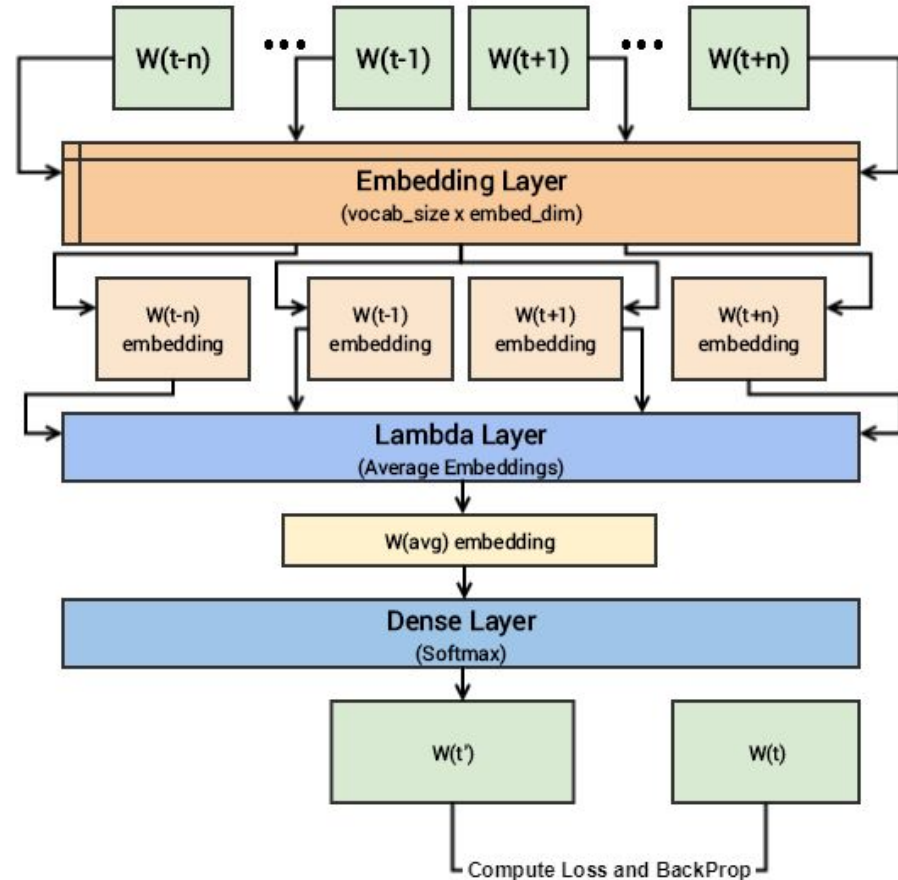
What is CBOW Model

The continuous bag-of-words (CBOW) model is a neural network for natural language processing tasks such as language translation and text classification. It is based on predicting a target word given the context of the surrounding words.

The CBOW model takes a window of surrounding words as input and tries to predict the target word in the center of the window.

How does CBOW work?

1. The context words are first passed as an input to an embedding layer (initialized with some random weights).
2. The word embeddings are then passed to a lambda layer where we average out the word embeddings.
3. We then pass these embeddings to a dense SoftMax layer that predicts our target word. We match this with our target word and compute the loss and then we perform backpropagation with each epoch to update the embedding layer in the process.



1. Build the corpus vocabulary

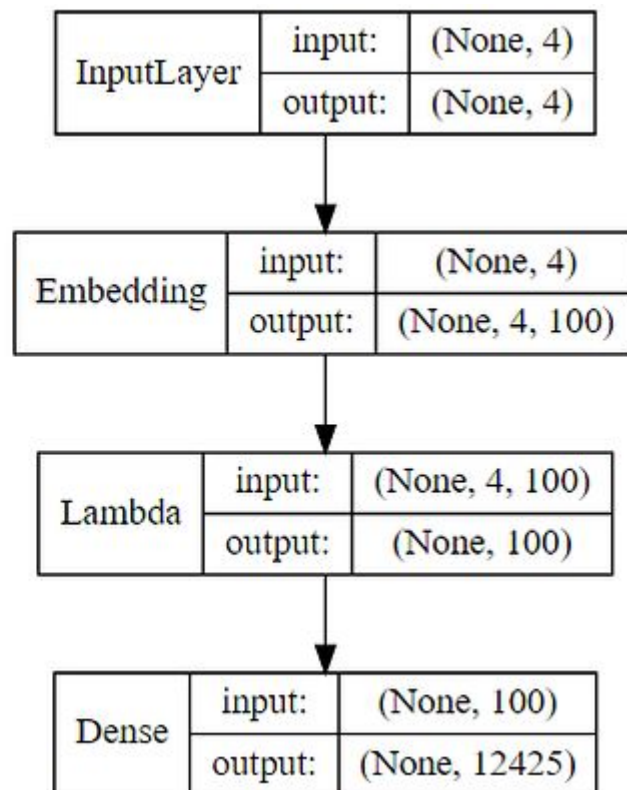
To start off, we will first build our corpus vocabulary where we extract out each unique word from our vocabulary and map a unique numeric identifier to it.

2. Build a CBOW (context, target) generator

```
Context (X): ['old','testament','james','bible'] -> Target (Y): king
Context (X): ['first','book','called','genesis'] -> Target(Y): moses
Context(X): ['beginning','god','heaven','earth'] -> Target(Y): created
Context (X): ['earth','without','void','darkness'] -> Target(Y): form
Context (X): ['without','form','darkness','upon'] -> Target(Y): void
Context (X): ['form', 'void', 'upon', 'face'] -> Target(Y): darkness
Context (X): ['void', 'darkness', 'face', 'deep'] -> Target(Y): upon
Context (X): ['spirit', 'god', 'upon', 'face'] -> Target (Y): moved
Context (X): ['god', 'moved', 'face', 'waters'] -> Target (Y): upon
Context (X): ['god', 'said', 'light', 'light'] -> Target (Y): let
Context (X): ['god', 'saw', 'good', 'god'] -> Target (Y): light
```

3. Build the CBOW model architecture

Layer (type)	Output Shape	Param #
=====		
embedding_1 (Embedding)	(None, 4, 100)	1242500
lambda_1 (Lambda)	(None, 100)	0
dense_1 (Dense)	(None, 12425)	1254925
=====		
Total params: 2,497,425		
Trainable params: 2,497,425		
Non-trainable params: 0		
=====		



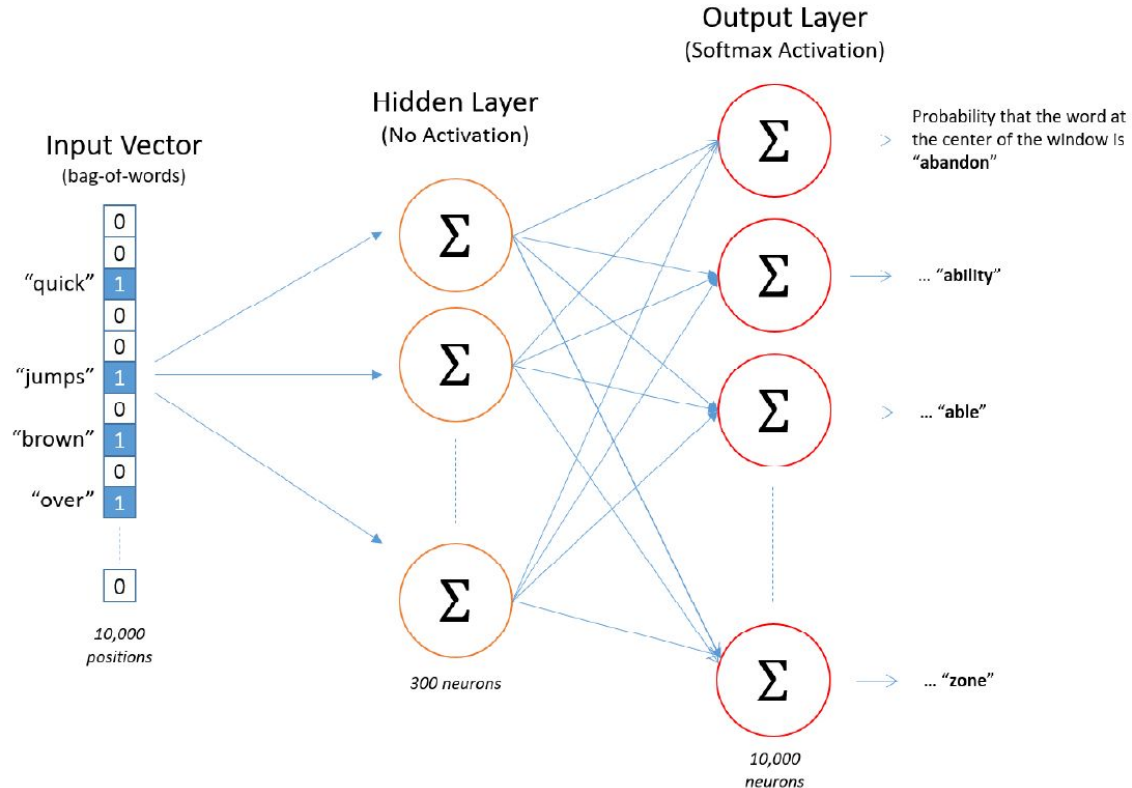
Let's Understand the intuition

1. Build the corpus vocabulary
2. Build a CBOW (context, target) generator
3. Build the CBOW model architecture
4. Train the Model
5. Get Word Embeddings

Let's code :)

1. Build the corpus vocabulary
2. Build a CBOW (context, target) generator
3. Build the CBOW model architecture
4. Train the Model
5. Get Word Embeddings

Let's do a numerical



"The king rules the land"

- Skip-gram: "king" $\rightarrow$? (predict context words)
- CBOW: ["the", "rules"] $\rightarrow$? (predict middle word)

Positive Training Samples

Sentence

The quick brown fox jumps over the lazy dog. →

Skip-Gram

(the, quick)
(the, brown)

CBOW

((quick, brown), the)

The quick brown fox jumps over the lazy dog. →

(quick, the)
(quick, brown)
(quick, fox)

((the, brown, fox), quick)

The quick brown fox jumps over the lazy dog. →

(brown, the)
(brown, quick)
(brown, fox)
(brown, jumps)

((the, quick, fox, jumps), brown)

The quick brown fox jumps over the lazy dog. →

(fox, quick)
(fox, brown)
(fox, jumps)
(fox, over)

((quick, brown, jumps, over), fox)

Skip-gram

Larger dataset

One method for computing embeddings: skip-gram with negative sampling, sometimes called SGNS. The skip-gram algorithm is one of two algorithms in a software package called word2vec, and so sometimes the algorithm is loosely referred to as word2vec

The intuition of skip-gram is:

1. Treat the target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples.
3. Use logistic regression to train a classifier to distinguish those two cases.
4. Use the learned weights as the embeddings.

How does Skip-gram work?

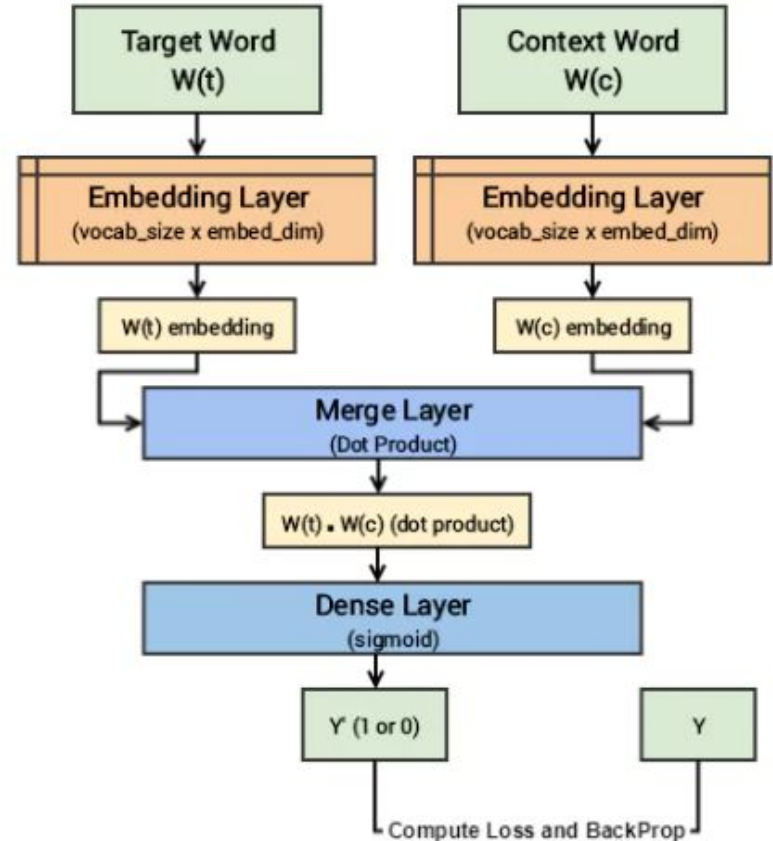
we feed the model pairs of (X, Y) where X is our input and Y is our label. This is done by creating positive input samples and negative input samples.

1. Positive Input Samples will have the training data in this form: [(target, context), 1] where the target is the target or centre word, context represents the surrounding context words, and label 1 indicates if it is a relevant pair.
2. Negative Input Samples will have the training data in the same form: [(target, random), 0]. randomly selected words are fed in along with the target words with a label of 0 indicating that it's an irrelevant pair.

How does Skip-gram work?

The following steps describe how the model works:

1. Both the target and context word pairs are passed to individual embedding layers from which we get dense word embeddings for each of these two words.
2. We then use a 'merge layer' to compute the dot product of these two embeddings and get the dot product value.
3. This dot product value is then sent to a dense sigmoid layer that outputs either 0 or 1.
4. The output is compared with the actual label and the loss is computed followed by backpropagation with each epoch to update the embedding layer in the process.



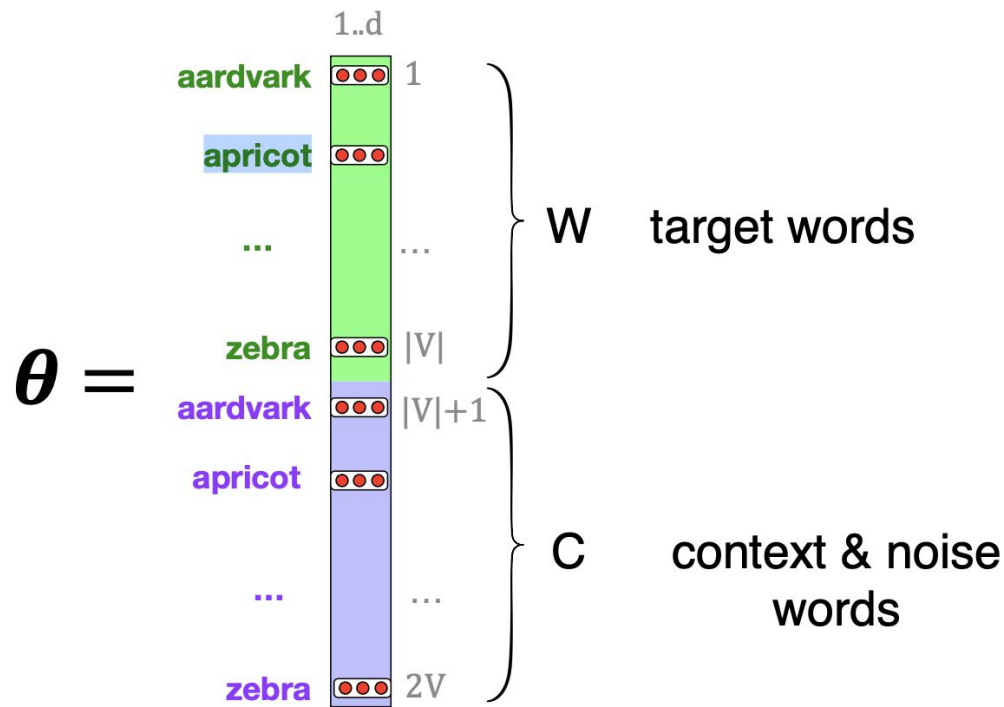


Figure 6.13 The embeddings learned by the skipgram model. The algorithm stores two embeddings for each word, the target embedding (sometimes called the input embedding) and the context embedding (sometimes called the output embedding). The parameter θ that the algorithm learns is thus a matrix of $2|V|$ vectors, each of dimension d , formed by concatenating two matrices, the target embeddings W and the context+noise embeddings C .

Learning skip-gram embeddings

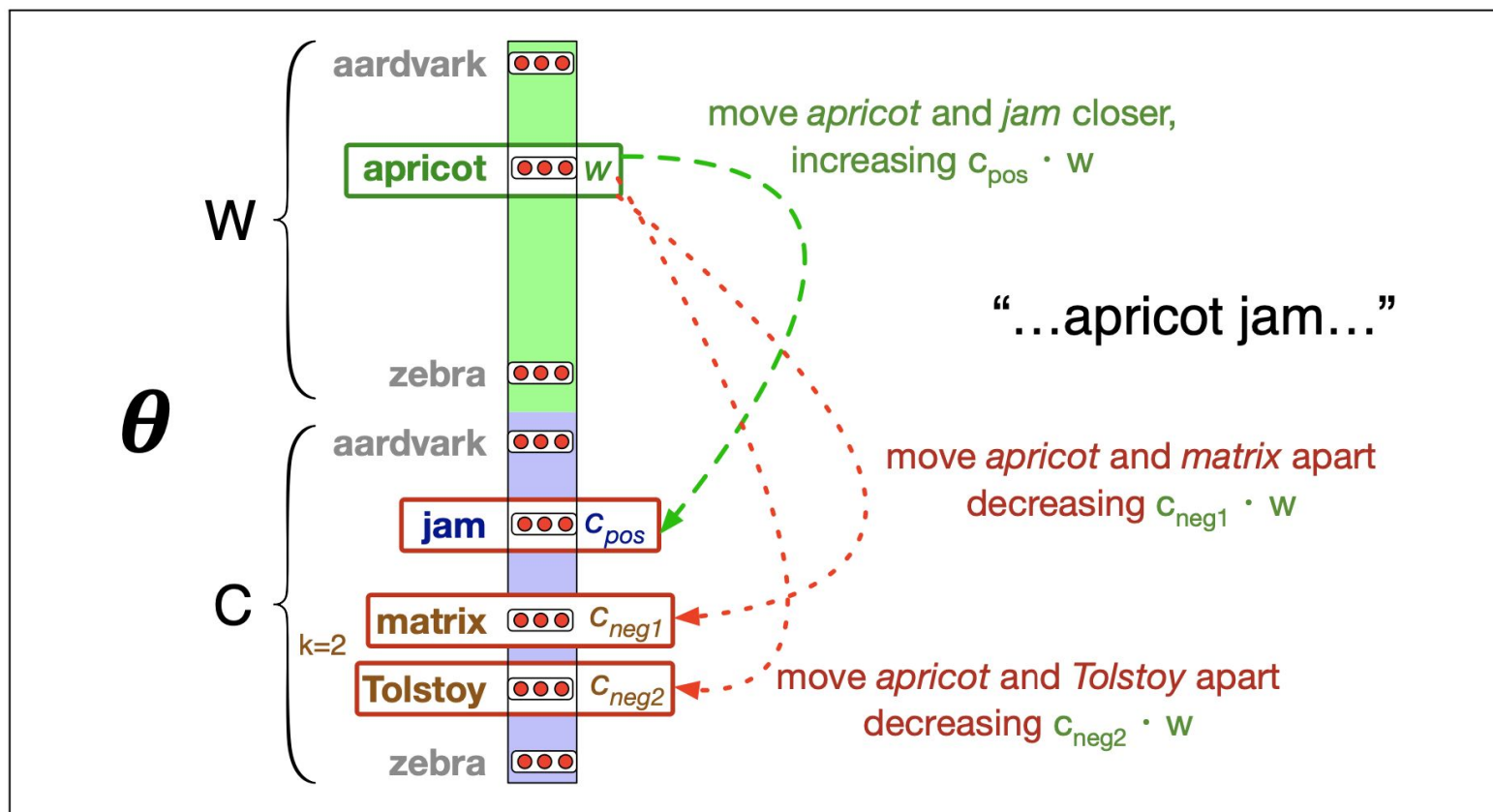


Figure 6.14 Intuition of one step of gradient descent. The skip-gram model tries to shift embeddings so the target embeddings (here for *apricot*) are closer to (have a higher dot product with) context embeddings for nearby words (here *jam*) and further from (lower dot product with) context embeddings for noise words that don't occur nearby (here *Tolstoy* and *matrix*).

Other kinds of static embeddings

CBOW

For smaller dataset

Increasing the training

Increasing the dimension of the vector

Increase Window size

The wide road shimmered in the hot sun.

`tf.keras.preprocessing.sequence.skipgrams`



(wide, road)	...	(road, shimmered)	(hot, sun)	...	(the, hot)
(2, 3)	...	(3, 4)	(6, 7)	...	(1, 6)

`tf.random.log_uniform_candidate_sampler`
(negative_samples = 4)



(wide, road)
(2, 3)



(wide, sun)	(wide, hot)	(wide, temperature)	(wide, code)
(2, 7)	(2, 6)	(2, 23)	(2, 2196)

concat and add label (pos:1/neg:0)



(wide, road)	(wide, sun)	(wide, hot)	(wide, temperature)	(wide, code)
(2, 3)	(2, 7)	(2, 6)	(2, 23)	(2, 2196)
1	0	0	0	0

build context words and labels for all vocab words



Word	Context words						Labels				
2	3	7	6	23	2196	⇒	1	0	0	0	0
23	12	6	94	17	1085	⇒	1	0	0	0	0
84	784	11	68	41	453	⇒	1	0	0	0	0
						⋮					
V	45	598	1	117	43	⇒	1	0	0	0	0

Evaluating Vector Models

1. Intrinsic evaluation → measures embedding quality directly, without using them in a real task.
 - a. Check whether the embeddings capture linguistic properties like similarity, analogy, and semantic consistency.
 - b. Fast and easy to compute.
 - c. Doesn't depend on external models.
 - d. May not reflect how well embeddings perform in real applications.
 - e. Word similarity benchmark
2. Extrinsic evaluation → measures how embeddings perform when used in downstream NLP tasks (e.g., sentiment analysis, NER).
 - a. Assess how embeddings improve the performance of a real NLP task.
 - b. Helps compare embeddings in actual applications.
 - c. Slow
 - d.

Examples

Intrinsic:

- You test analogies $\rightarrow \text{king} - \text{man} + \text{woman} \approx \text{queen}$ ✓
- High correlation with WordSim-353 human scores ✓

Extrinsic:

- You use those embeddings in a sentiment classifier.
- Accuracy improves from 82% $\rightarrow$ 86% ✓

Code

https://colab.research.google.com/drive/1i7zB7fCu-FvR5bACiKVAvbcCA02_pb41?usp=sharing

Text Classification

Binary Classification

Multiclass Classification

Multilabel Classification

Naive Bayes

Reading Task

- Read reserach paper “Efficient Estimation of Word Representations in Vector Space”. It is uploaded on team.
- And “Distributed Representations of Words and Phrases and their Compositionality” paper

References

- Chapter # 06 of speech and language book (uploaded on teams)
- <https://www.analyticsvidhya.com/blog/2021/07/word2vec-for-word-embeddings-a-beginners-guide/>
- Research paper “Efficient Estimation of Word Representations in Vector Space” - uploaded on teams
- <https://www.kdnuggets.com/2018/04/implementing-deep-learning-methods-feature-engineering-text-data-cbow.html>
- Research paper “Intrinsic and Extrinsic Evaluations of Word Embeddings”